

LOW-LEVEL NETWORK PACKET INSPECTION & PROTOCOL ANALYSIS

*TCP Handshake Behaviour, Packet Layer Structure & Clear-Text Authentication Exposure —
Isolated Lab Environment*

Prepared By	Uzair Ahmed
Student ID	B4/963
Target Scope	Loopback interface (127.0.0.1) — isolated Kali Linux VM
Assessment Type	Packet-level protocol analysis and clear-text credential exposure testing
Tools Used	Wireshark, pyftplib, Python Flask, VMware Kali Linux
Assessment Window	21 August 2026
Report Classification	Internal / Lab Use — Internship Task 3

1. Executive Summary

This assessment was carried out as part of Task 3 of the internship program: a low-level, packet-by-packet inspection of network traffic using Wireshark, aimed at understanding TCP connection behaviour and identifying clear-text authentication exposure in unencrypted application protocols.

Two controlled services were stood up on an isolated Kali Linux VM (VMware) for the exercise: an FTP server (pyftplib) on port 21 and a Flask-based HTTP login form on port 8080. All traffic was captured on the Loopback (lo) interface using Wireshark. The exercise confirmed that both FTP and HTTP transmit authentication credentials in fully readable plain text, that TCP's three-way handshake behaviour is identical and protocol-independent across both services, and that packet capture at the application layer requires no special network position when credentials are unencrypted — anyone with visibility of the wire can read them directly.

No unauthorized data exfiltration to external hosts was observed; all captured conversations were confined to the loopback address (127.0.0.1), consistent with the isolated nature of the lab setup.

2. Scope & Methodology

Scope

Testing was restricted to locally-hosted services on a single isolated Kali Linux virtual machine. All client and server processes ran on the same host; traffic was captured exclusively on the Loopback (lo) interface. No external or production systems were involved.

Tools & Setup

- Wireshark — live packet capture and protocol dissection on interface lo
- pyftplib — lightweight FTP server (port 21) used to generate authentic FTP control-channel traffic
- Python Flask — minimal HTTP login form (port 8080) used to generate authentic HTTP POST traffic
- Standard FTP client (ftp localhost) and Firefox browser used as clients

Process

- Start each service and begin a live Wireshark capture on the loopback interface before initiating any connection.
- Connect as a client (FTP login / HTTP form submission) to generate a fresh TCP handshake and application-layer exchange.
- Filter captured traffic by protocol and by tcp.port to isolate the three-way handshake for each service.
- Expand the protocol layers of representative packets (Ethernet → IP → TCP → Application) to examine header fields.
- Use Statistics → Conversations to review traffic volume and endpoints for signs of unauthorized data movement.

3. Services Under Test

The table below summarizes the two locally-hosted services exercised during this assessment.

Port	Protocol	Service	Notes
21	FTP	pyftplib 2.2.0	Plain-text authentication; USER/PASS visible in capture
8080	HTTP	Flask dev server	Plain-text POST body; login form served without TLS

4. TCP Three-Way Handshake Analysis

Wireshark was filtered to isolate the connection setup for each service. In both cases the standard TCP three-way handshake (SYN → SYN, ACK → ACK) was observed immediately before any application data was exchanged, confirming that TCP handshake behaviour is independent of the application-layer protocol running above it.

4.1 FTP Handshake (tcp.port == 21)

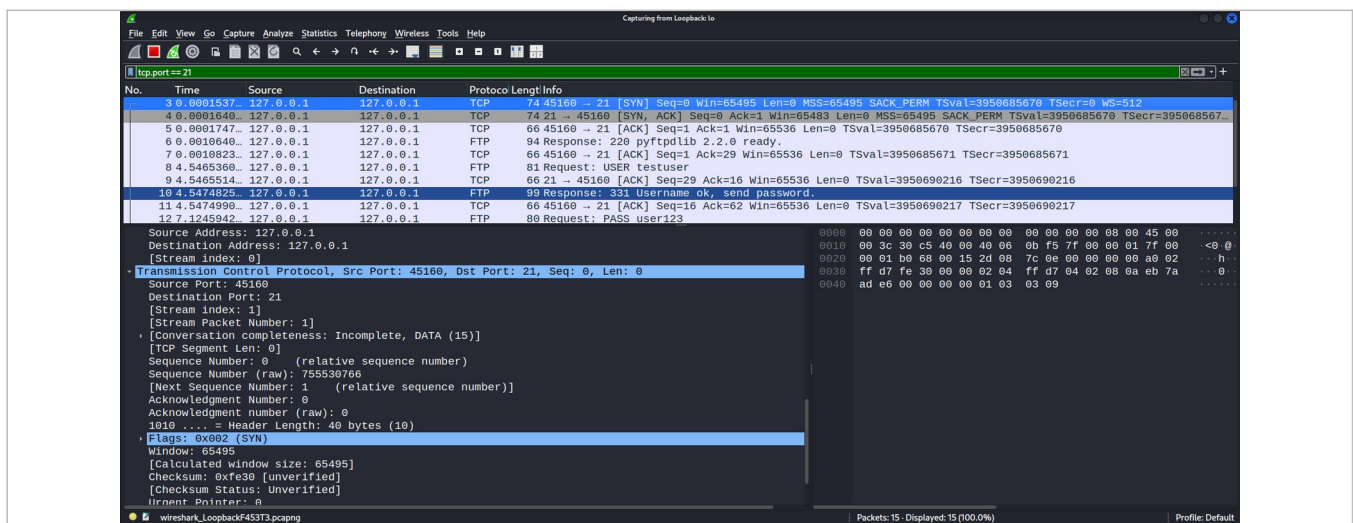


Figure 1 — TCP three-way handshake (packets 3–5) preceding the FTP session on port 21, followed immediately by the FTP banner and USER command.

4.2 FTP Client Session (Terminal View)

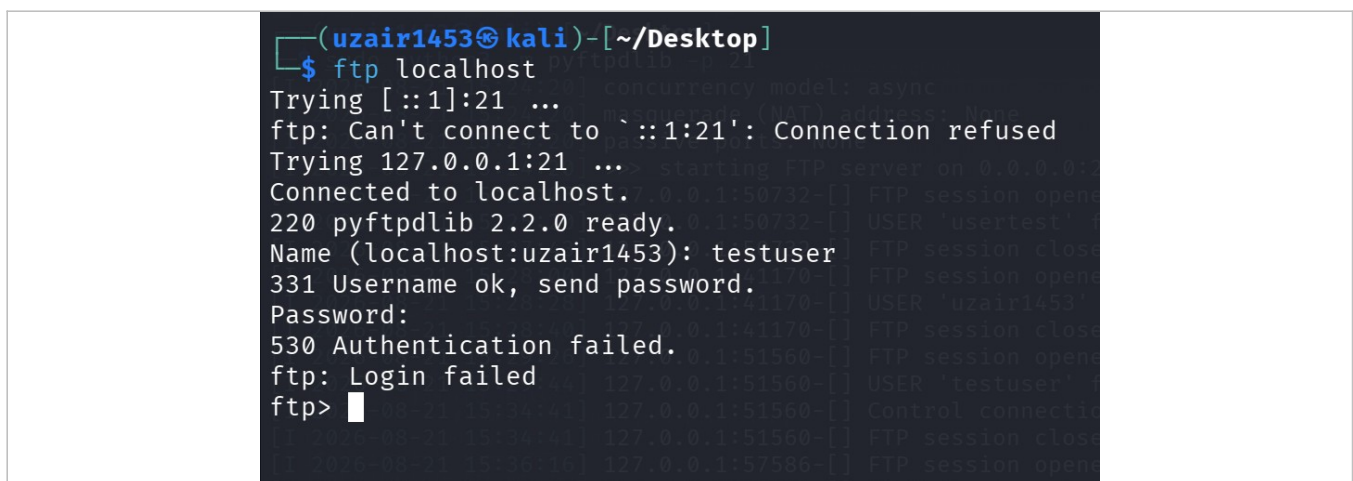


Figure 2 — FTP client (ftp localhost) initiating the control connection, corresponding to the handshake captured in Figure 1.

5. Packet Stream & Protocol Layer Breakdown

Individual packets were expanded in Wireshark to examine each encapsulation layer — Ethernet II (data link), Internet Protocol (network), Transmission Control Protocol (transport), and File Transfer Protocol (application). The FTP USER command packet below illustrates this layering, with the ASCII payload “USER testuser” visible directly in the hex dump pane.

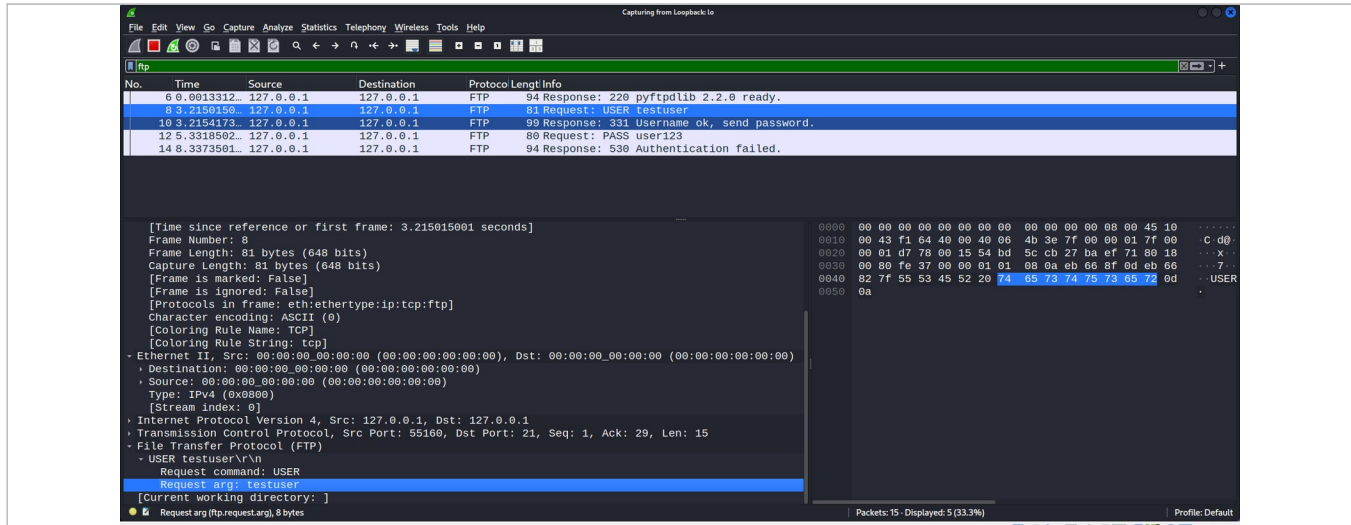


Figure 3 — Expanded protocol layers for an FTP USER packet: Ethernet II → IPv4 → TCP → FTP, with the plain-text argument “testuser” highlighted in the hex/ASCII view.

The FTP server log (below) confirms the same session lifecycle from the server side — connection open, USER command received, authentication outcome, and disconnect — matching what was independently observed at the packet level.

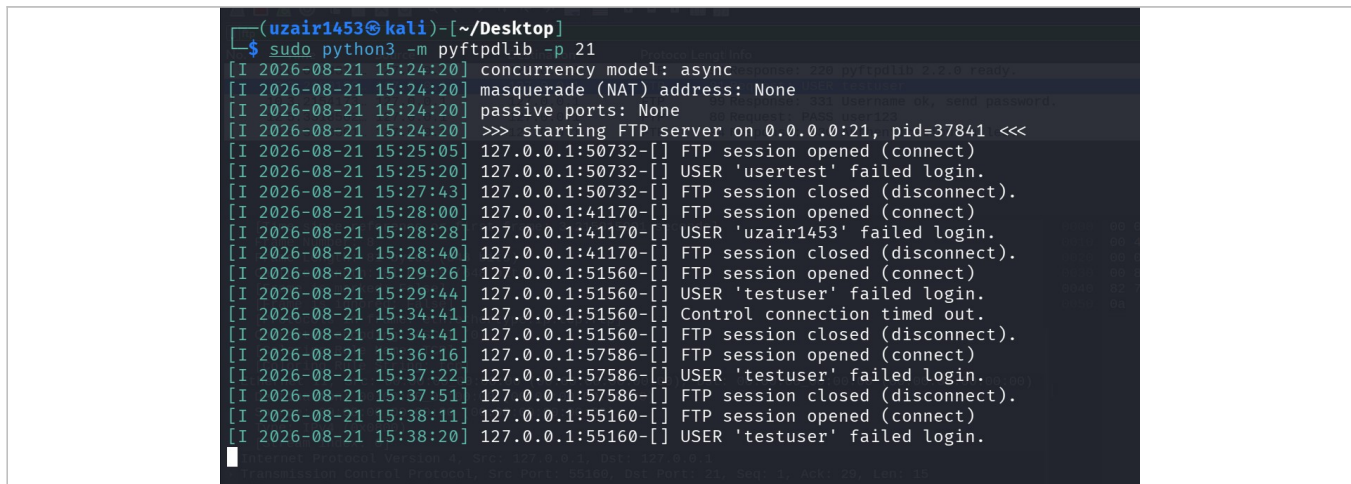


Figure 4 — pyftplib server-side log showing repeated FTP session opens, USER attempts, and failed logins.

6. Unencrypted Protocol Vulnerability — FTP

Filtering on the ftp protocol isolated the authentication exchange. The USER and PASS commands were transmitted as plain ASCII text with no encryption, meaning any party with visibility of the network segment (or, as here, the loopback interface) can read the submitted credentials directly, regardless of whether the login attempt ultimately succeeds or fails.

Observed request: USER testuser, followed by PASS user123 — both fully readable in the Wireshark packet list and hex pane (Figure 3).

7. Unencrypted Protocol Vulnerability — HTTP

A minimal Flask login form was served over plain HTTP on port 8080 to demonstrate the same class of exposure in a web context. The Flask development server log below confirms the sequence of GET and POST requests generated while testing the form.

```
(uzair1453@kali) - [~/Desktop/http_test]
$ python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://192.168.0.105:8080
Press CTRL+C to quit
127.0.0.1 - - [21/Aug/2026 16:34:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [21/Aug/2026 16:34:09] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [21/Aug/2026 16:51:26] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [21/Aug/2026 16:51:54] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [21/Aug/2026 16:52:06] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [21/Aug/2026 16:52:11] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [21/Aug/2026 16:52:41] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [21/Aug/2026 16:56:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [21/Aug/2026 16:56:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [21/Aug/2026 16:56:32] "POST / HTTP/1.1" 200 -
```

Figure 5 — Flask development server log showing GET requests (page loads) and POST requests (form submissions) on port 8080.

The login form itself masks the password field on screen using dots, giving the visual impression of protection:

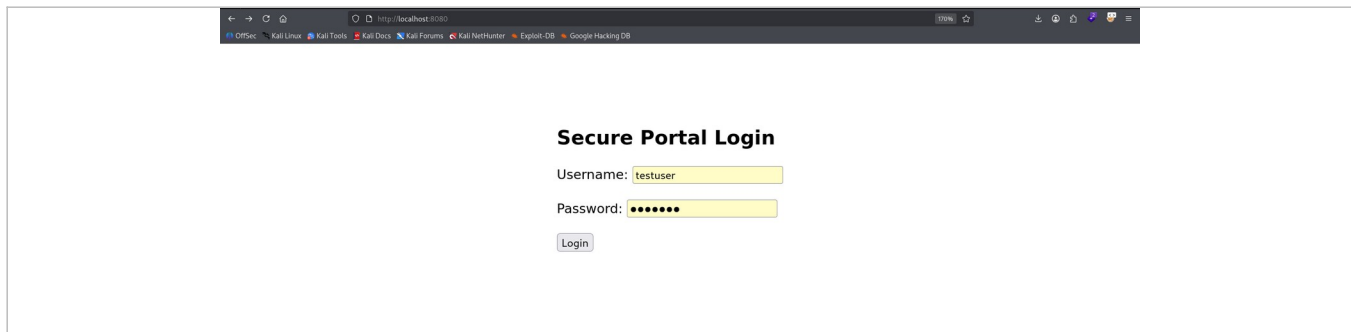


Figure 6 — Browser view of the “Secure Portal Login” form; the password field is masked in the UI.

However, because the form is submitted over HTTP rather than HTTPS, the POST request body is transmitted without any transport-layer encryption. Following the TCP stream in Wireshark reveals both submitted values in full clear text:

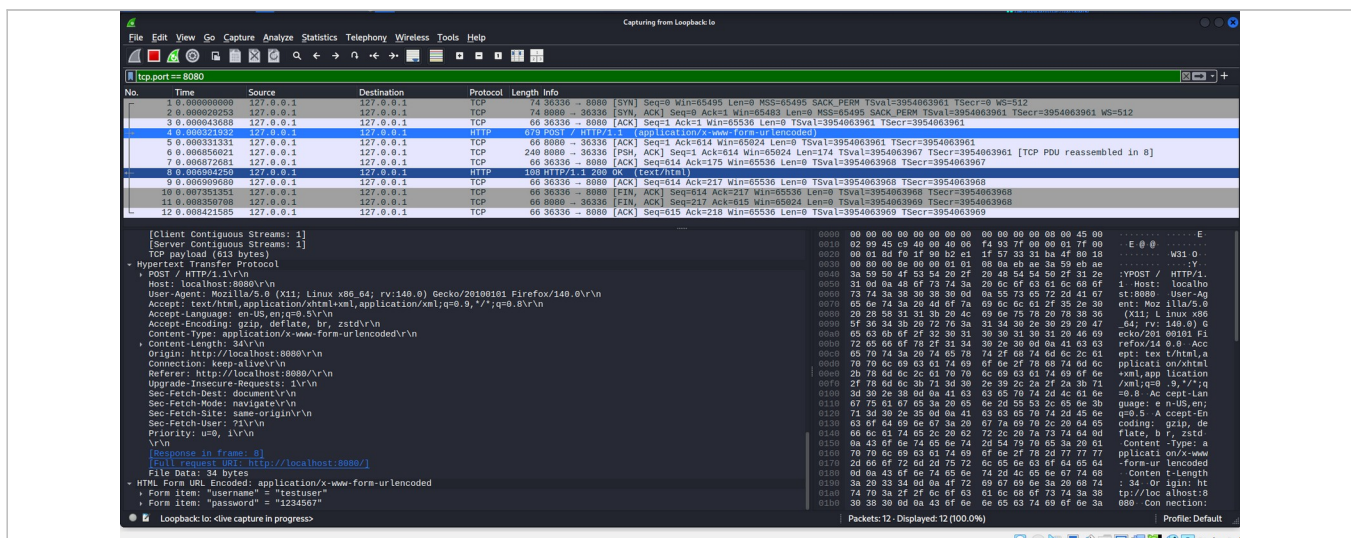


Figure 7 — Captured HTTP POST request showing the TCP handshake, the full request headers, and the decoded form data — username = “testuser”, password = “1234567” — despite the password field appearing masked in the browser.

This confirms an important finding for the report: client-side field masking is a UI convenience only and provides no actual confidentiality. The only effective control against this exposure is transport-layer encryption (HTTPS/TLS).

8. Traffic & Exfiltration Trend Analysis

Statistics → Conversations was used to review the volume and direction of traffic for each captured session, looking for any indication of data being sent to an unexpected or external destination.

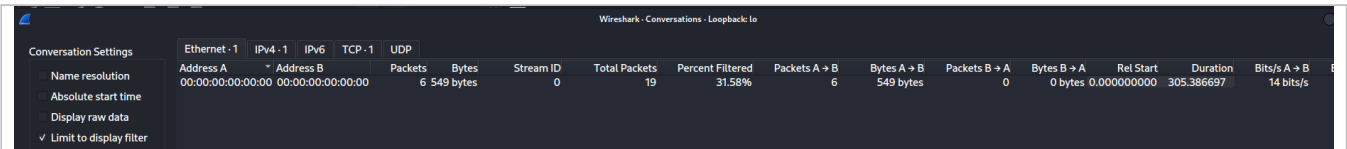


Figure 8 — FTP session conversation summary: 6 packets, 549 bytes, entirely between 127.0.0.1 and itself.

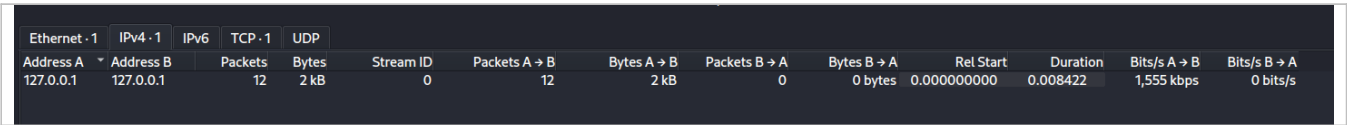


Figure 9 — HTTP session, IPv4 view: all traffic confined to 127.0.0.1 ↔ 127.0.0.1.

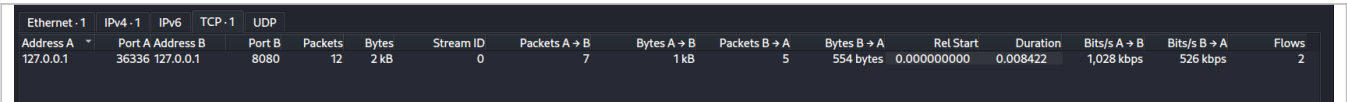


Figure 10 — HTTP session, TCP view: client (port 36336) sent 1 kB to the server (port 8080), which returned 554 bytes — consistent with a single login POST and its response.

In every case, source and destination addresses were identical (127.0.0.1), and total traffic volumes were small and consistent with the manual actions performed (a handful of page loads and form submissions). No unauthorized or unexplained data transfer to an external address was observed in this controlled environment.

9. Bonus Observation — Anomalous Login Pattern

While reviewing the FTP server log (Figure 4), multiple failed login attempts were recorded from the same loopback client across several short-lived sessions, each closing shortly after a failed USER/PASS exchange. In a production network, this repeated failed-login pattern in a short time window is a recognizable anomaly signature consistent with a brute-force or credential-stuffing attempt, and would typically trigger rate-limiting or account lockout controls.

10. Findings Summary

Severity	Area	Observation	Recommendation
HIGH	FTP (port 21)	USER and PASS commands transmitted in fully readable plain text; credentials visible directly in the hex payload.	Replace FTP with SFTP or FTPS so the control channel is encrypted end-to-end.
HIGH	HTTP (port 8080)	POST body containing username and password captured unencrypted despite the password field being	Serve the login form over HTTPS/TLS; never accept credentials over plain HTTP.

Severity	Area	Observation	Recommendation
		masked in the browser UI.	
MEDIUM	Authentication logic	Repeated failed FTP login attempts from the same client in short succession resemble a brute-force pattern.	Implement account lockout / rate-limiting after repeated failed login attempts.
LOW	Traffic scope	All captured conversations were confined to 127.0.0.1 (loopback); no external destination IPs were observed.	No action required in this lab context; in production, monitor Conversations for unexpected external destinations.

11. Recommendations

- Replace FTP with an encrypted equivalent (SFTP or FTPS) so authentication and file transfer are protected in transit.
- Serve any web-based login form exclusively over HTTPS/TLS; never accept credentials over plain HTTP, even for internal or test portals.
- Do not rely on client-side field masking (password dots) as a security control — it is cosmetic only.
- Apply rate-limiting or account lockout after repeated failed login attempts to mitigate brute-force style access patterns.
- Periodically review Conversations/endpoint statistics in production environments to flag unexpected external destinations as a low-cost exfiltration check.

12. Conclusion

This exercise demonstrated, using real captured traffic rather than theoretical description, that TCP's three-way handshake is a consistent, protocol-independent foundation underlying both FTP and HTTP connections, and that both of these unencrypted application protocols expose authentication credentials in fully readable plain text — confirmed directly in the packet hex/ASCII data and via Wireshark's Follow TCP Stream feature. Traffic volume analysis confirmed no unauthorized exfiltration occurred within the scope of this isolated lab environment. The findings reinforce a single governing principle: confidentiality of credentials in transit depends entirely on transport-layer encryption (TLS), not on application-level conventions such as password masking or command structure.